Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

Path planning and trajectory optimization based on an improved RRT algorithm

Chaofan Teng

2727626344@qq.com

Shenyang Aerospace University

Luping Wang

Shenyang Aerospace University

Research Article

Keywords: Path planning, RRT, Minimum snap, Trajectory optimization, Time reallocation

Posted Date: May 10th, 2024

DOI: https://doi.org/10.21203/rs.3.rs-4320725/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

Path planning and trajectory optimization based on an improved RRT algorithm

Author details:

Chaofan Teng:Email：2727626344@qq.com, Affiliation :College of Aeronautics and Astronautics, Shenyang University of Aeronautics and Astronautics, Shenyang, Liaoning, China

Luping Wang:Email：293529957@qq.com, Affiliation :Engineering Training Center, Shenyang University of Aeronautics and Astronautics, Shenyang, Liaoning, China

Abstract: This study presents an enhanced motion planning and trajectory optimization algorithm for Rapidly-exploring Random Trees (RRT), addressing the issues of excessive randomness, unnecessary path segments, and discontinuities in the trajectory that are evident in conventional RRT path planning algorithms. Initially, a dynamic sampling strategy was developed to minimize the search's randomness by managing the generation location of random points. Subsequently, the artificial potential field method was incorporated into the RRT algorithm to enable nodes to account for the influence of obstacles and target points during expansion; thus, mitigating the search's aimlessness. Redundant sections were then eliminated, and the path distance was reduced, all the while maintaining a safe distance from obstacles. Finally, by employing the minimum snap method in conjunction with flight corridors and time reallocation, the trajectory was rendered smooth and continuous, ensuring that physical quantities remained within acceptable limits without encountering obstacles. The comparative simulation results demonstrate that the proposed algorithm markedly decreases the number of nodes extended during the path search and reduces the search path length in comparison to the traditional RRT.

Keywords: Path planning, RRT, Minimum snap, Trajectory optimization, Time reallocation

1 Introduction

With the rapid advancement of robotics technology and the automotive industry, autonomous robot motion and autonomous driving technologies have garnered extensive attention. Due to the significant impact of path planning and trajectory optimization on the reliability and safety of mobile robots and autonomous vehicles, path planning has increasingly become a subject of focus [1] [2] [3]. The objective of path planning is to identify the optimal or suboptimal route from a start point to an endpoint, ensuring the path adheres to performance indicators such as distance cost and operational time, and avoids collision with obstacles [4] [5] [6] [7].

Presently, there is a vast body of research on path planning, which can generally be categorized into three types: sampling-based algorithms, graph search-based algorithms, and artificial intelligence algorithms [8]. Graph search-based algorithms are deterministic search methods that find the optimal path by establishing a cost function and other metrics. However, these algorithms are primarily suited for solving discretization problems due to limitations in image representation. Because they require a global map search, these algorithms demand extensive computation, especially on structurally complex maps where the computational complexity can increase significantly, impacting their performance. Furthermore, in constrained global maps, such algorithms may become stuck in local optima, resulting in high time costs. Examples of graph search-based algorithms include the Dijkstra algorithm [10] and the A* algorithm [11]. Artificial intelligence algorithms can enhance performance through continuous learning and optimization. Nonetheless, AI-based path searching necessitates substantial experimental data [9] and extended computation times. This category includes algorithms like the genetic algorithm [12], UP2TA algorithm [13], and ant colony algorithm [14].

Sampling-based algorithms are probabilistic and operate by randomly sampling the global map to generate a random tree of nodes from the starting point. As the number of samples increases, the random tree gradually approaches the endpoint, eventually reaching the target. These algorithms have minimal constraints and are well-suited for nonholonomic constraint problems. The RRT algorithm is highly representative of this category, known for its effectiveness and robustness in path planning research [15]. Currently, various optimized RRT algorithms have been developed. For example, Jiankun Wang et al. [18] applied a Gaussian mixture model to simulate the natural growth of trees, reducing the number of nodes generated by avoiding repeated searches in the same direction. JR and Lavalle introduced an RRT Connect algorithm for bidirectional exploration, enhancing the speed of path search [19]. Other improvements include the Hybrid RRT [16] and Fast RRT [17].

The RRT algorithm, being a probability-based discrete search method, exhibits several limitations during its application process. The search process of this algorithm is characterized by high randomness, leading to a significant presence of duplicate regions, which diminishes the search's efficiency. Furthermore, the algorithm does not consider map information when determining the direction for the expansion of the search tree, resulting in suboptimal use of available information. The generation of a path with turning points is dependent upon the positions of random points, introducing a several redundant nodes along the path. The emergence of additional nodes

consequently extends the path length. The path delineated by RRT and its enhanced versions consists of line segments connecting multiple nodes, rendering the path discontinuous. Such discontinuities necessitate trajectory optimization, as autonomous vehicles or mobile robots are incapable of navigating these paths.

Addressing the high randomness inherent to RRT has become a pivotal area of research. Luan Tiantian et al. [20] introduced a dynamic variable sampling area algorithm to circumvent the issue of becoming ensnared within specific regions. Kang Bohan et al. [21] developed a strategy that adjusts the step size according to the environmental complexity, employing shorter step sizes in obstacle-dense areas and larger ones in less obstructed spaces to augment search efficiency. Tu Haiyan et al. positioned sub-nodes within narrow areas to expedite search times in intricate environments. While these enhancements have improved RRT's performance in particular contexts and settings, their general applicability remains limited. Moreover, these methods have not fully considered the impact of existing nodes on the map on the search tree's expansion.

The issue of redundant paths in RRT limits its application in current engineering projects, prompting extensive research. Karaman [26] proposed the RRT* algorithm, an asymptotically optimal variant that refines the initial path yielded by RRT through continuous random point sampling, allowing the path to converge towards an approximate optimal route. Liao Bin et al. 25 employed nodes based on triangle inequalities to derive shorter paths. Nonetheless, the RRT* algorithm lacks precise targeting in selecting extension points, leading to slower convergence rates in complex spaces [22-23] and necessitating high computational effort.

To render the path continuous, Liu Hui et al. [27] employed third-order B-splines to smooth the trajectory while accounting for the robot's motion curvature constraint. The minimum snap method has also found widespread use in trajectory optimization 2829. These techniques transform the trajectory into a continuous curve and restrict the trajectory's position through the properties of B-splines and flight corridors, respectively. However, constraints on the robot's speed and acceleration were not imposed, potentially exceeding the capabilities of the corresponding actuators in practical scenarios.

In light of the discussions above, this article proposes an improved RRT algorithm and optimizes the resulting path. Initially, a dynamic sampling area method was designed to minimize duplicate areas during the search. Subsequently, an artificial potential field guidance strategy was introduced, incorporating the artificial potential field method to lessen the randomness of search tree expansion. The initial path was then optimized by transforming redundant segments into straight lines, yielding a shorter suboptimal path. Finally, the minimum snap method, coupled with flight corridors and time reallocation, was applied for trajectory optimization to enhance the trajectory's practical applicability.

The structure of this article is organized as follows: Section 2 elaborates on the fundamental principles of the RRT algorithm. Section 3 introduces the main enhancements to the RRT algorithm proposed in this study. Section 4 discusses the minimum snap method. Section 5 presents simulation validations of the improved algorithm in two scenarios, and Section 6 concludes the study.

2 RRT algorithm principle

The RRT algorithm is a probability-based path search algorithm. It employs the starting point as the root node, randomly selects points within the configuration space, and finally extends from the child node with a certain step length. If there is no collision with obstacles, the newly expanded node is added to the search tree. The search continues until a node reaches the goal. After the search ends, the search tree

is backtracked to obtain a path that starts at the beginning and ends at the destination. The pseudocode of the RRT algorithm is shown in Table 1:

Table 1 Pseudocode of the RRT algorithm

Algorithm 1: ***RRT***	
Input:	*M, start, goal*
Output:	*L*
1	*T.init(start)*
2	**for** *i = 1 to n* **do**
3	*rand←Sample(M);*
4	*near←Near(brand, T);*
6	*new←Step(rand, near,StepSize);*
7	**If** *CollisionFree(x_{new})* **then**
8	*T.addNode(x_{new});*
9	*T.rewire();*

The input M is the map information. Starting from $start$. $goal$ is the endpoint. The output L is the path from the starting point to the target point. T represents the initialization of the search tree. $rand$ is a random sampling point. $near$ is the child node closest to the random point on the search tree. new represents the newly generated child node. After obtaining the new node new, CollisionFree is used to detect whether the search tree collides with obstacles. If no collision occurs, the new node is added to the search tree through an addNode. Finally, the path is obtained through rewired backtracking.

Since the RRT algorithm is a probability-based search method, there is randomness as well as blindness in its search process. The above deficiencies will lead to lower search efficiency, as well as the possibility of getting into a local deadlock and failing to find a path [24]. Moreover, the paths obtained by the RRT algorithm do not have continuity, and trajectories need to be optimized [25].

3 Improved RRT algorithm

To mitigate the RRT algorithm's challenges associated with non-directed global sampling and the proliferation of extraneous nodes within the search tree, which impair search efficiency, a dynamic sampling region strategy is implemented. This strategy excludes already searched regions and takes into account the influence of previously generated nodes during the random point generation process, thereby reducing search repetition. Furthermore, an artificial potential field-oriented sub-node expansion direction strategy is applied. This approach increases the directedness of search tree expansion by integrating the navigable areas of the environment, obstacles, and target points into the determination of sub-node expansion directions. Additionally, to overcome the issue of redundant path nodes and the generation of numerous detour paths by the RRT algorithm, resulting in increased overall path distances, the child nodes of each path point are re-selected to shorten the path length. However, as some paths become too proximate to obstacles following the re-selection of sub-nodes, complicating subsequent trajectory optimization, certain path points are re-planned. This re-planning increases the separation between the trajectory and obstacles, thereby simplifying the optimization process.

3.1 Dynamic Sampling Region Strategy by Removing Search Regions

The RRT algorithm assesses the attainment of the goal point by determining if the newly generated node falls within a specified vicinity of the endpoint. This can be reinterpreted as verifying whether the goal point is situated within a certain radius around the new node, effectively defining the node's search range.

In the standard RRT algorithm, the direction in which child nodes expand is solely influenced by the location of random points, leading to their positions being determined randomly. This method of search results in a significant repetition of search areas, thereby extending the duration of the search process. As depicted in Fig. 1, the black dots represent nodes that are already part of the search tree. X_{rand} indicates a random point. X_{new} stands for a newly generated node. It is assumed that a

circular area of radius r is drawn at the center of each node as the node search area. The blue area in Fig. 1 represents the search area of the nodes that already exist in the search tree, and the red area represents the search area of the newly generated node. when the newly generated node develops toward the already searched area. The search area of the newly emerged child nodes has most of the area in the already searched range. The overlapping part belongs to the area of the repeated search, which results in the efficiency of the search being reduced.

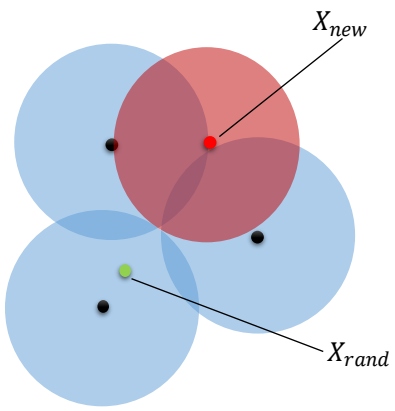


Figure 1 RRT search area overlap phenomenon

To avoid generating too many duplicate regions. A dynamic sampling region strategy is proposed that removes the search region. The generation of duplicate regions is reduced by restricting the sampling points outside the already explored regions. In Fig. 2, the black points represent the child nodes in the search tree. and the blue regions represent the searched ranges. X_{rand1} indicates the random point location in the unrestricted random point generation area. X_{new} is the position of the newly generated child node. Random points are resampled when they are within $R=\{x \in X \mid \|x-P_i\|<r\}$. X_{rand1} is within the searched area and therefore does not meet the requirement to obtain X_{rand2}. By resampling, X_{rand2} in the unsearched range becomes compliant. The search tree was expanded to X_{new}.

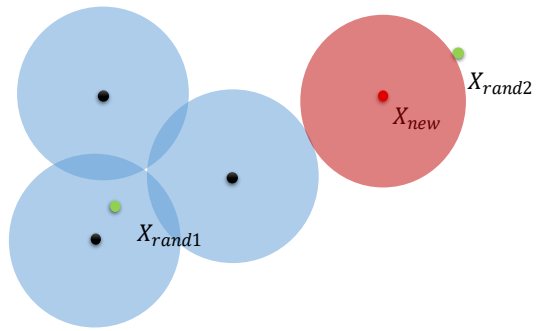


Fig. 2 Dynamic sampling area

3.2 Artificial Potential Field-Oriented Node Expansion Strategy

The generation of new nodes in the RRT algorithm involves extending a fixed distance towards random points from the parent node. However, this method lacks the ability to adjust the expansion direction based on surrounding obstacles, potentially leading to search stagnation. To address this issue and enhance the directedness of search tree expansion, we propose the artificial potential field-oriented node expansion strategy.

This strategy integrates the artificial potential field method, designating regions surrounding obstacles as high-potential field areas, while areas devoid of obstacles and, including the target point are designated as low-potential field regions. During the generation of new nodes, the high-potential field region exerts a repulsive force on the new node, whereas the low-potential field region exerts an attractive force. Consequently, the expansion direction of nodes tends to steer away from obstacles and towards the endpoint, imbuing the search tree extension with a greater sense of purpose.

When environmental factors are not considered for node expansion, as shown in Fig. 3(a). The new node is generated at X_{new}, and

its generation position cannot prevent the search tree from being close to the obstacle, which can easily lead to reciprocal motion around the obstacle.

With the addition of the artificial potential field, the direction of influence of the potential field is shown in Fig. 3(b). where r_{blank} is the direction of attraction of the blank area, the area without obstacles, to the new node. r_{obs} is the direction of repulsion of the obstacle area to the new node. r_{goal} is the attraction direction of the target point to the new node.

Summing the above effects due to environmental factors and random points generated by multiplying the coefficients in the four directions, the final vector direction obtained is the direction of the expansion of the final child node, and its direction expression is:

$$r = p_r \cdot r_r + p_{blank} \cdot r_{blank} + p_{obs} \cdot r_{obs} + p_{goal} \cdot r_{goal} \tag{1.1}$$

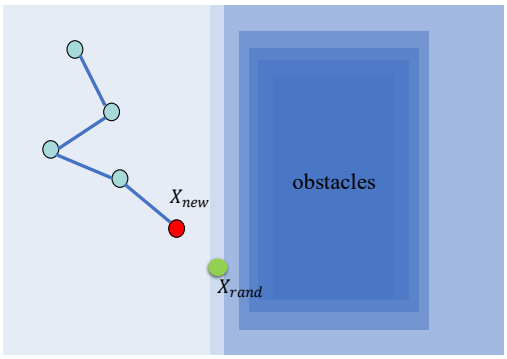
where p_r is the random point correlation coefficient, p_{blank} is the blank area attraction correlation coefficient, p_{obs} is the obstacle repulsion correlation coefficient, and p_{goal} is the target point correlation coefficient.

When the child node is distant from the target point, incorporating the influence of the target point into the expansion direction of the new node might disrupt its trajectory, potentially trapping it in a localized search state and diminishing search efficiency. To circumvent this issue, we establish a threshold to prevent such occurrences. This threshold is defined as the distance from the parent node to the target point. When this condition is met, the influence of the target point is factored into the direction of the child node extension. The expression for the direction of the child node is as follows:

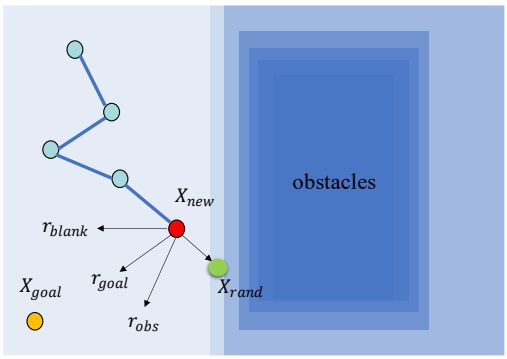
$$r = \begin{cases} p_r \cdot r_r + p_{blank} \cdot r_{blank} + p_{obs} \cdot r_{obs} + p_{goal} \cdot r_{goal}, \rho \le s \\ p_r \cdot r_r + p_{blank} \cdot r_{blank} + p_{obs} \cdot r_{obs} \qquad , \rho > s \end{cases} \tag{3.1}$$

where ρ is the distance from the parent node to the target point. s is the threshold.

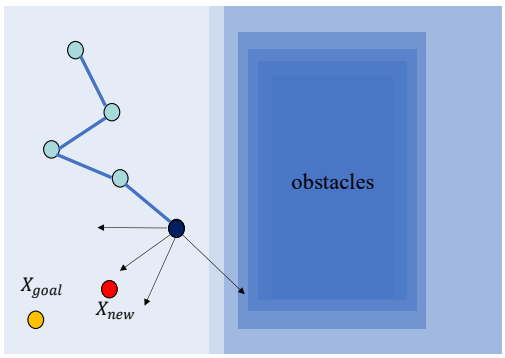
After the above effects, a new node is finally produced at location X_{new}, as shown in Fig. 3(c).



(a) Direction of random point expansion when environmental factors are not considered



(b) Effect of environmental factors on the direction of random point expansion



(c) Final Expansion Position of Nodes under Combined Influence

Fig. 3 Impact of environmental factors on the

expansion of new nodes

By integrating environmental factors into the expansion of the search tree, new nodes exhibit a predisposition to veer away from obstacles and gravitate towards obstacle-free regions and the goal point. This inclination diminishes the likelihood of the search process encountering a deadlock.

3.3 Path Simplification

As the path produced by the RRT algorithm comprises line segments connecting multiple random points, it inherently contains numerous redundant nodes and segments, resulting in an extended total path length. To mitigate this, path simplification is imperative. Redundant paths and nodes are eliminated by re-evaluating the route using the original path points and conducting collision detection to shorten the path distance.

As shown in Fig. 4, X_i is the node location. $path_{pre}$ represents the initial path generated by the RRT algorithm. $path_{new}$ indicates the optimized path. The black area is the obstacle. Assuming that the path is optimized from X_1, connect X_1 and X_i to form a new path until the newly generated path collides with an obstacle. In Fig. 4, the path formed by the link between X_1 and X_7 will collide with the obstacles. The path formed by X_1 and X_6 will not collide with the obstacles, so the path formed by the link between X_1 and X_6 is selected as the optimization path. Meanwhile, in the next section of the optimization path, X_6 will be used as the starting point to start the new optimization.

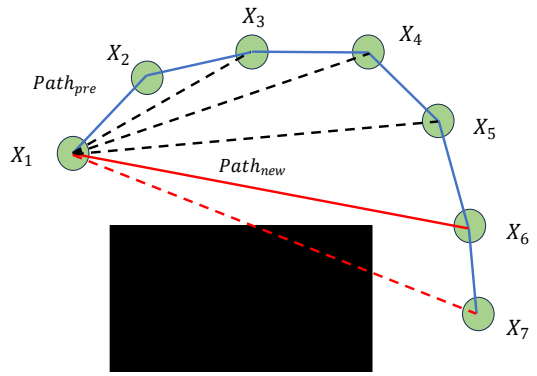


Figure 4 Simplified schematic of the path

Following the initial simplification of the path, certain segments may draw closer to obstacles, potentially complicating subsequent trajectory optimization. To address this issue, it is crucial to maintain a safe distance between the path and obstacles. For segments that approach obstacles too closely, new nodes are inserted along the original path. These new nodes are strategically placed to ensure that the distance between them and obstacles falls within an acceptable range, thereby guaranteeing that the entire path remains sufficiently distanced from obstacles. As shown in Fig. 5, P_{new} is a newly inserted point, and the rest of the points are represented the same as in Fig. 4. A new node P_{new} is added at the location where the distance between path $path_{new}$ and the obstacle is too close, and the distance between point P_{new} and the obstacle is within a reasonable safe distance. Paths X_1 - P_{new} - X_6 are used as optimized paths.

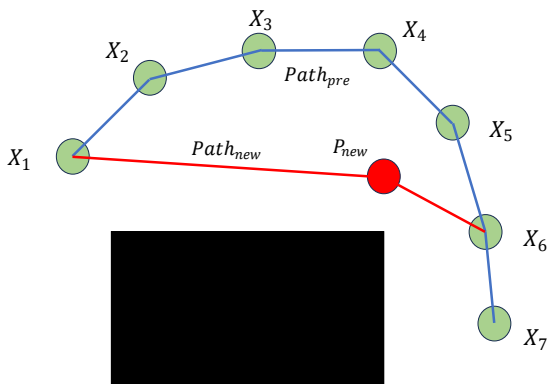


Fig. 5 Rearranging nodes to make path principle obstacles

After simplifying the path, the total length of the path was shortened while maintaining a reasonable distance between the obstacles and the path.

4 Minimum Snap Polynomial Trajectory Optimization

The trajectory yielded by the RRT algorithm lacks continuity and smoothness, rendering it unsuitable for practical use by carriers. Hence, trajectory optimization is indispensable. In this study, we employ the minimum snap method to optimize the trajectory. Moreover, to prevent path deviation during optimization from resulting in collisions with obstacles, we utilize flight corridors to establish inequality constraints, confining the trajectory within a specified range. Furthermore, to optimize the actual physical parameters, we redistribute the time allocated to each segment of the running duration.

4.1 Establishment of Optimization Function

The path obtained by improving the RRT algorithm has a trajectory that can be considered a polynomial function in different directions, which is expressed as:

$$p(t) = p_0 + p_1 t + p_2 t^2 \ldots + p_n t^n \tag{4.1}$$

A longer trajectory may be more difficult to express as a polynomial. The trajectory can be divided into multiple segments by time, with each segment represented by a polynomial curve:

$$p(t) = \begin{cases} [1, t, t^2, \ldots, t^n] \cdot p_1 t_0 \le t \le t_1 \\ [1, t, t^2, \ldots, t^n] \cdot p_2 t_1 \le t \le t_2 \\ \ldots \\ [1, t, t^2, \ldots, t^n] \cdot p_k t_{K-1} \le t \le t_K \end{cases} \tag{4.2}$$

where K is the total number of segments of the trajectory. n indicates the highest power of the polynomial. p are the polynomial coefficients. $p_{m,n}$ represents the n -th coefficient of the polynomial trajectory of segment m.

Since snap is a fourth-order derivative of position, the optimization function can be expressed as:

$$\int_0^T (\mathrm{p}^{(4)}(\mathrm{t}))^2 dt = \sum \int_{t_{i-1}}^{t_i} (f^4(t))^2 dt \tag{4.3}$$

where t_i is the moment of arrival at the end of the i-th trajectory.

Equation (4.3) is expressed by the following matrix:

$$\sum_{i=1}^{k} p^T \int_{t_{i-1}}^{t_i} [0,0,0,0,24,\ldots,\frac{n!}{(n-4!)} t^{n-4}]^T [0,0,0,0,24,\ldots,\frac{n!}{(n-4!)} t^{n-4}] p dt = \sum_{t=1}^{k} p^T Q_i p \tag{4.4}$$

Of which Q_i are:

$$\mathbf{Q_i} = \begin{bmatrix} 0_{4\times4} & 0_{4\times(n-3)} \\ 0_{(n-3)\times4} & \frac{n!m!}{(n-4)!(m-4)!(n+m-7)} (t_j^{n+m-7} - t_{j-1}^{n+m-7}) \end{bmatrix} \tag{4.5}$$

4.2 Establishing Equality Constraints

During the optimization process utilizing the minimum snap trajectory, it is imperative to satisfy both smoothness and continuity constraints, thereby establishing two equation constraints.

The smoothness constraint dictates that the connection point between the end of the preceding segment of the trajectory and the subsequent segment should adhere to

predefined specifications. The expression for the smoothness constraint is as follows:

$$f_i^{(k)}(t_i) = x_i^{(k)} \tag{4.6}$$

where k is the k th order derivative. i represents the i-th trajectory. t_i is the moment when the end of the i-th trajectory is reached. x_i indicates the position of the end of the i-th trajectory.

The continuity constraint requires that the position, velocity, acceleration, and derivative of acceleration should be equal at each trajectory intersection. The expression for continuity is provided below. :

$$f_i^{(k)}(t_i) = f_{i+1}^{(k)}(t_i) \tag{4.7}$$

Using polynomials, equation (4.7) can be converted to:

$$\sum_{n\geq k} \frac{n!}{(n-k)!} t_i^{n-k} p_{i.n} = x_i^{(k)} \tag{4.8}$$

$$\sum_{n\geq k} \frac{n!}{(n-k)!} t_i^{n-k} p_{n,i} - \sum_{m\geq k} \frac{m!}{(m-k)!} t_i^{m-k} p_{m,i+1} = 0 \tag{4.9}$$

Using the matrices, equation (4.9) can be represented as:

$$[\mathbf{A}_i \quad -\mathbf{A}_{i+1}] \begin{bmatrix} \mathbf{p}_i \\ \mathbf{p}_{i+1} \end{bmatrix} = 0 \tag{4.10}$$

The matrix representation of the final equation constraint is:

$$\mathbf{A}_{\mathbf{eq}}\mathbf{p} = \mathbf{b}_{\mathbf{eq}} \tag{4.11}$$

4.3 Establishment of Inequality Constraints for Dynamic Corridor Cells

Optimizing the trajectory solely based on equation constraints ensures continuity and smoothness of path points. However, without additional measures, points along the path may stray from the intended trajectory, potentially leading to collisions. To address this, it is essential to establish inequality constraints to confine the path within predefined boundaries.

The concept behind constructing inequality constraints is to confine the midpoint between each path point within a flight corridor. Therefore, the inequality restrictions for each trajectory point can be formulated as follows:

$$\begin{cases} x_{\min} \leq p_i(t)_x \leq x_{\max} \ , i \in (1, I), t \in [0, t_i] \\ y_{\min} \leq p_i(t)_y \leq y_{\max} \ , i \in (1, I), t \in [0, t_i] \end{cases} \tag{4.12}$$

where $x_{\min}, x_{\max}, y_{\min}, y_{\max}$ is the boundary of the corridor grid. l is the total number of intermediate points. t_j is the moment at which the end of the j-th segment of the trajectory is reached. p_i is the i-th intermediate point.

Furthermore, considering the varying distances from obstacles at different path points, it is essential to adjust the constraint range accordingly. For instance, employing a larger corridor limit may render the flight corridor inadequate for effectively constraining the trajectory. Conversely, opting for smaller corridors as constraints may intensify the overlap between the optimized and original paths, weakening the optimization outcomes. Therefore, corridors of different sizes should be utilized to constrain the path based on its location. In regions with restricted access, smaller surface connections should be designated as corridors, whereas in areas with more extensive access, larger areas should serve as corridors. The inequality constraints are expressed as follows:

$$\mathbf{b}_{\mathbf{lieq}} \leq \mathbf{A}_{\mathbf{ieq}}\mathbf{p} \leq \mathbf{b}_{\mathbf{rieq}} \tag{4.13}$$

4.4 Reallocation of Time

After incorporating the flight corridor, the path is constrained only in terms of position, lacking restrictions on physical quantities such as speed and acceleration. Consequently, certain locations may experience elevated values of speed and acceleration, as depicted in Fig. 6, exceeding the capabilities of the actual actuator. Moreover, in areas with shorter

distances, speeds may remain lower, prolonging the overall travel time. Therefore, it becomes imperative to readjust the time allocation to modulate the values of speed and acceleration in the relevant segments of the path, thereby mitigating discrepancies and optimizing performance.

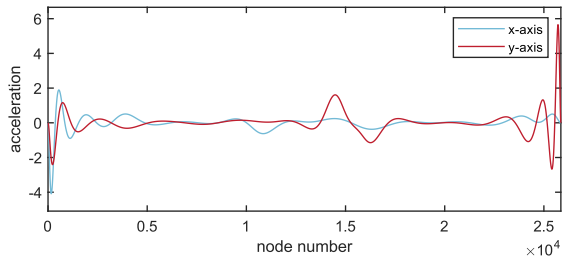


Fig. 6 Acceleration with undistributed time

Time reallocation can be scaled up or down by scaling variables such as speed relative to a reasonable range. The expression for time reallocation is:

$$t_{new} = k \cdot t_{pre} \qquad (4.14)$$

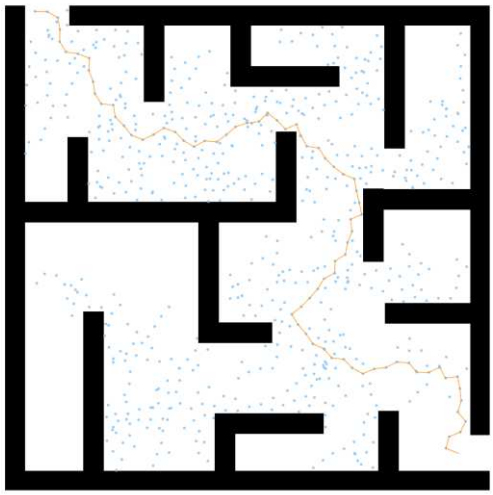
where t_{new} is the time after reallocation. k is the percentage of change. t_{pre} indicates the initial allocation of time. Then, trajectory optimization is performed by a minimum snap combined with a flight corridor approach.

5 Simulation verification

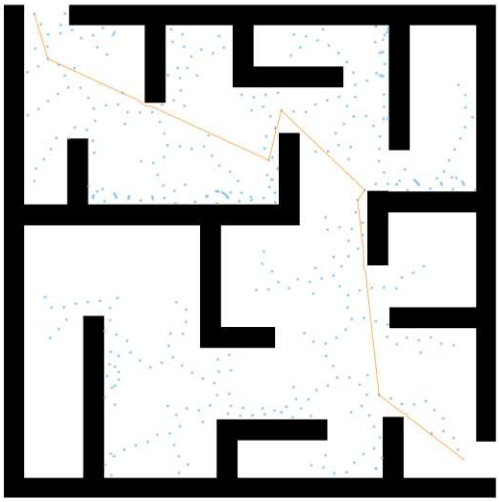
To verify the effect of the algorithm improvement, simulations were performed in the software MATLAB2021b, and experiments were conducted in two scenarios.

5.1 Simulation Comparison of Path Planning

The path planning simulation results are shown in Fig. 7 and Fig. 8, where the blue points are the search nodes of the RRT search tree. The orange line segments are the final planned paths. Fig. 7a and Fig. 8 show the results obtained by the RRT algorithm, and Fig. 7b shows the results of the paths obtained by the improved RRT algorithm.

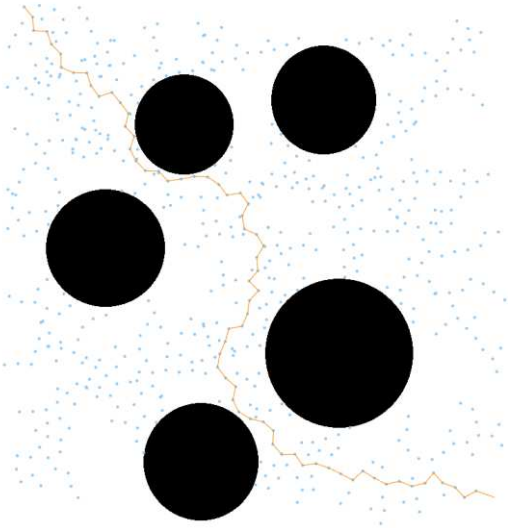


a Simulation results of the RRT algorithm

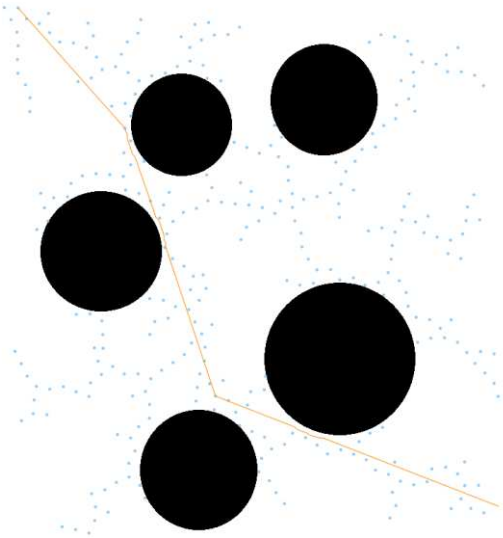


b Simulation results of the improved RRT algorithm

Fig. 7 Scene 1 path planning simulation results



a Simulation results of the RRT algorithm



b Simulation results of the improved RRT algorithm

Fig. 8 Scene 2 path planning simulation results

Comparison of the paths generated by the RRT algorithm and the improved RRT algorithm in the two scenarios. The improved RRT algorithm has the following characteristics:

(1)After incorporating the position of existing nodes into the generation of random points, each node is more evenly distributed, enabling a larger search range with the same number of nodes. This enhancement improves the search efficiency of each node compared to the standard RRT algorithm.

(2)Additionally, the artificial potential field-oriented node expansion strategy results in fewer nodes in the search tree when traversing extensive passable areas, thereby accelerating search speed in such regions.

(3)Moreover, the final path obtained after initial path optimization eliminates numerous redundant paths, shortening the overall path length and reducing the number of trajectory turning points.

In Scenario 1, the number of extended search points required by the improved RRT algorithm is compared with that of the standard RRT algorithm. Experimental results depicted in Fig. 9 illustrate that the red curve represents the number of extended search points for the path searched by the RRT algorithm, while the blue curve represents the number of extended search points for the path searched by the improved RRT algorithm.

After 20 trials, the RRT algorithm requires an average expansion of 586 search points to find the path. Conversely, the improved RRT algorithm requires an average expansion of 385 search points to find the path. Thus, the improved algorithm exhibits a 34.3% optimization over the original.

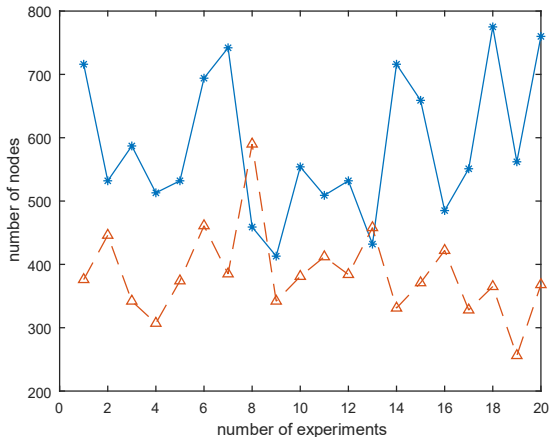


Fig. 9 Number of search points generated by searching into the path

Moreover, the final path lengths obtained by the RRT algorithm and the improved RRT algorithm are compared. The path lengths obtained by the RRT algorithm and the improved RRT algorithm after 20 experiments are shown in Fig. 10, where the blue folded lines are the lengths of the paths of the RRT algorithm. The red folded lines are the lengths of the paths of the improved RRT algorithm.

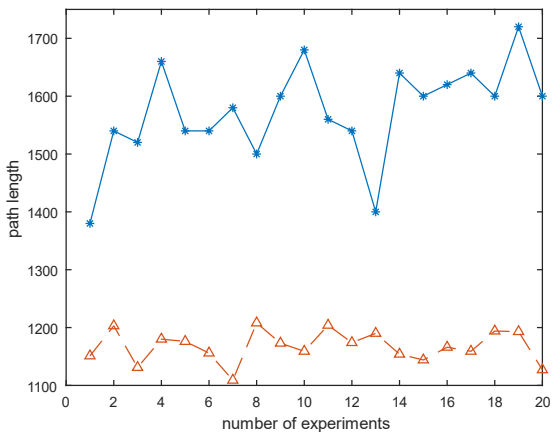


Figure 10 Generating path lengths

The average path length obtained by the RRT algorithm is 1573, the average path length obtained by the improved RRT algorithm is

1168, and the improved algorithm reduces the distance by 25.8%.

5.2 Simulation of trajectory optimization

The optimized trajectory is depicted in Fig. 11 and Fig. 13, where the blue rectangular region represents the range of inequality constraints. The red solid line illustrates the minimum snap optimized trajectory. Notably, the optimized trajectory successfully navigates from the starting point to the endpoint without encountering collisions, ensuring continuity throughout the trajectory.

The optimization outcomes of the trajectories in the first scenario are presented in Figure 11:

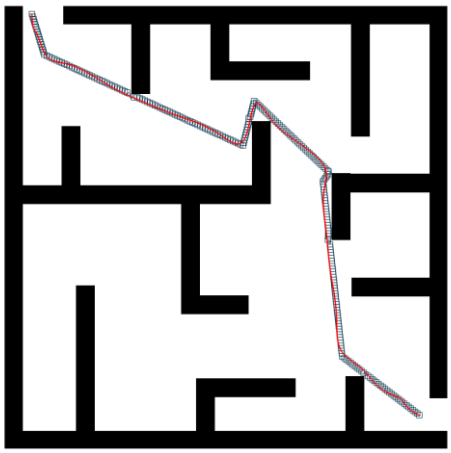
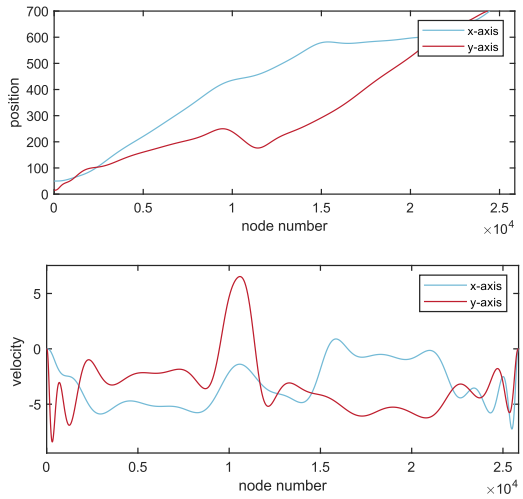


Fig. 11 Scene 1 trajectory optimization results

The position, velocity, and acceleration profiles after trajectory optimization are shown in Figure 12.



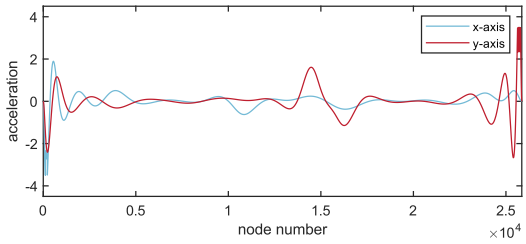


Fig. 12 Position, velocity and acceleration curves in Scene 1

The results of trajectory optimization in the second scenario are shown in Fig. 13:

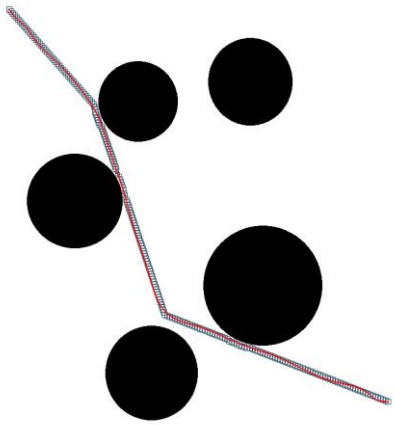


Fig. 13 Scene 2 trajectory optimization results

The position, velocity, and acceleration curves are shown in Figure 14.

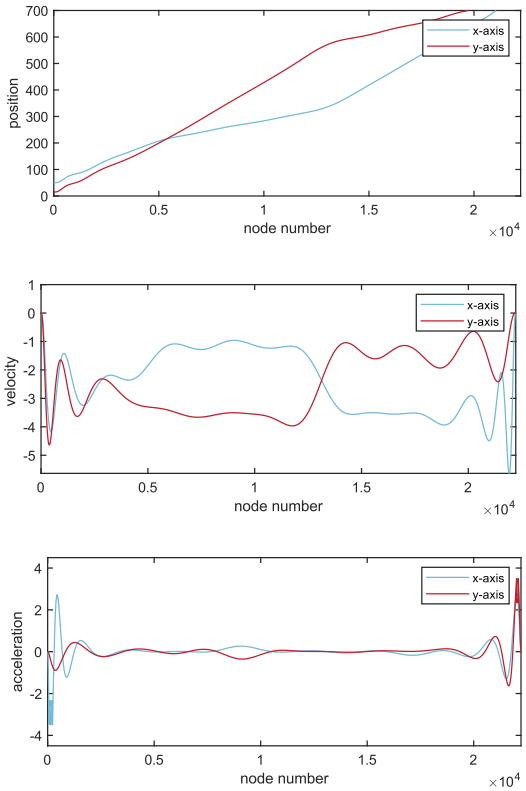


Fig. 14 Position, velocity, and acceleration curves for Scene 2

After trajectory optimization, the trajectory becomes a smooth and continuous curve while not colliding with obstacles, and the peak acceleration is reduced after redistribution of time.

6 Conclusion

This paper introduces several enhancements to the RRT algorithm to address its limitations. Firstly, a dynamic sampling area strategy is implemented to mitigate the high repetition rate of random point generation, thereby reducing both the number of random point generation iterations and the overall path search time. Secondly, an artificial potential field-guided sub-node expansion strategy is introduced to impart purposefulness to the expansion of the search tree. This strategy aims to streamline the search process and improve efficiency. Additionally, path redundancy is quickly addressed through optimization techniques, resulting in the generation of suboptimal paths with fewer computations. Finally, trajectory optimization is achieved by combining the minimum snap method with flight corridor constraints and time reassignment.

Following these improvements, the enhanced algorithm demonstrates notable advantages over the traditional RRT algorithm in simulation experiments:

(1) The improved algorithm significantly reduces the number of child nodes generated during path search. Compared to the traditional RRT algorithm, this enhancement enhances search efficiency by 34.3% and shortens paths by 25.8%. This reduction in child nodes not only simplifies the search process but also minimizes unnecessary computation, thereby accelerating overall search speed.

(2) To improve the algorithm, we considered the influence of environmental factors and endpoint positions on the expansion direction of child nodes. Through this method, the extension of the search tree becomes more purposeful because it can guide the search process based on environmental information and the location of the target point. This strategy helps to reduce unnecessary search branches and further improve the search efficiency.

(3) In terms of optimizing the trajectory, the concepts of minimum snap and a flight corridor are combined. The minimum snap ensures smooth continuity of the trajectory, while the flight corridor ensures that the trajectory does not collide with any obstacles when moving between the starting and ending points. In addition, the running time of each segment should be replanned to avoid excessive speed and acceleration. This optimization strategy not only smoothens the trajectory but also ensures the safety and stability of the robot during its movement.

ACKNOWLEDGEMENTS

This study is supported by the Innovation and Entrepreneurship Training Program for Chinese College Students(202310143002).

Data available on request from the authors.

REFERENCES

1. O. Salzman and D. Halperin, “Asymptotically Near-Optimal RRT for Fast, High-Quality Motion Planning,” Transactions on Robotics, vol. 32, no. 3, pp. 473–483, Jun. 2016.
2. M. Hirayama, J. Guivant, J. Katupitiya, and M. Whitty, “Path planning for autonomous bulldozers,” Mechatronics, vol. 58, pp. 20–38, Apr. 2019.
3. X. Lan and Stefano Di Cairano, “Continuous curvature path planning for semi-autonomous vehicle maneuvers using RRT,” CiteSeer X (The Pennsylvania State University), Jul. 2015.
4. Q. Chai and Y. Wang, “RJ-RRT: Improved RRT for Path Planning in Narrow Passages,” Applied Sciences, vol. 12, no. 23, p. 12033, Nov. 2022.

5. G. Tang, C. Tang, C. Claramunt, X. Hu, and P. Zhou, “Geometric A-Star Algorithm: An Improved A-Star Algorithm for AGV Path Planning in a Port Environment,” IEEE Access, vol. 9, pp. 59196–59210, 2021.
6. Z. Zhang, D. Wu, J. Gu, and F. Li, “A Path-Planning Strategy for Unmanned Surface Vehicles Based on an Adaptive Hybrid Dynamic Stepsize and Target Attractive Force-RRT Algorithm,” Journal of Marine Science and Engineering, vol. 7, no. 5, p. 132, May 2019.
7. Y. Shi, Q. Li, S. Bu, J. Yang, and L. Zhu, “Research on Intelligent Vehicle Path Planning Based on Rapidly-Exploring Random Tree,” Mathematical Problems in Engineering, vol. 2020, p. e5910503, Mar. 2020.
8. Z. Wang, Y. LU, Y. ZHOU, S. WEN, and C. ZHOU, “Research and Prospect on Autonomous Path Planning for Lunar Based Equipment,” Chinese Space Science and Technology, vol. 43, no. 11–24.
9. W. Cui and F. Zhu, “A Review of Research on Path Planning Algorithms for Robot Navigation.,” Computer Engineering and Applications, vol. 59, no. 10–20, Aug. 2023.
10. J. Mao et al., “Review of research and applications on path planning technology for power inspection robots,” Control and Decision, vol. 38.
11. P. Ji, “Research on Path Planning by Integrating Improved A* Algorithm and Dynamic Window Method,” Journal of System Simulation.
12. P. Du, X. He, H. Cao, S. Garg, G. Kaddoum, and M. M. Hassan, “AI-based energy-efficient path planning of multiple logistics UAVs in intelligent transportation systems,” Computer Communications, May 2023.
13. P. Muñoz, M. D. R-Moreno, and D. F. Barrero, “Unified framework for path-planning and task-planning for autonomous robots,” Robotics and Autonomous Systems, vol. 82, pp. 1–14, Aug. 2016.
14. C. Miao, G. Chen, C. Yan, and Y. Wu, “Path planning optimization of indoor mobile robot based on adaptive ant colony algorithm,” Computers & Industrial Engineering, vol. 156, p. 107230, Jun. 2021.
15. P. Zhao et al., “Dynamic RRT: Fast Feasible Path Planning in Randomly Distributed Obstacle Environments,” Journal of Intelligent and Robotic Systems, vol. 107, no. 4, Mar. 2023.
16. R. Mashayekhi, M. Y. I. Idris, M. H. Anisi, and I. Ahmedy, “Hybrid RRT: A Semi-Dual-Tree RRT-Based Motion Planner,” IEEE Access, vol. 8, pp. 18658–18668, 2020.
17. Z. Wu, Z. Meng, W. Zhao, and Z. Wu, “Fast-RRT: A RRT-Based Optimal Path Finding Method,” Applied Sciences, vol. 11, no. 24, p. 11777, Dec. 2021.
18. J. Wang, W. Chi, M. Shao, and M. Q.-H. . Meng, “Finding a High-Quality Initial Solution for the RRTs Algorithms in 2D Environments,” Robotica, vol. 37, no. 10, pp. 1677–1694, Feb. 2019.
19. J. Chen, Y. Zhao, and X. Xu, “Improved RRT-Connect Based Path Planning Algorithm for Mobile Robots,” IEEE Access, vol. 9, pp. 145988–145999, 2021.
20. T. Luan, H. Wang, X. Sun, and C. Lv, “Path planning of unmanned vehicle based on dynamic variable sampling area RRT,” Control and Decision.
21. B. Kang and J. Huang, “Variable step size rapidly-exploring random tree (RRT) path planning algorithms and simulation of a mobile robot based on environment complexity,” Journal of Beijing University of Chemical Technology.

22. J. Qi, Q. Yuan, C. Wang, X. Du, F. Du, and A. Ren, “Path planning and collision avoidance based on the RRT*FN framework for a robotic manipulator in various scenarios,” Complex and Intelligent Systems, vol. 9, no. 7475–7494.
23. I. Noreen, A. Khan, H. Ryu, N. L. Doh, and Z. Habib, “Optimal path planning in cluttered environment using RRT*-AB,” Intelligent Service Robotics, vol. 11, no. 1, pp. 41–52, Aug. 2017.
24. Y. Guo, X. Liu, Q. Jia, X. Liu, and W. Zhang, “HPO-RRT*: a sampling-based algorithm for UAV real-time path planning in a dynamic environment,” Complex & Intelligent Systems, Jun. 2023.
25. L. Yuan, J. Zhao, W. Li, and J. Hou, “Improved Informed-RRT* Based Path Planning and Trajectory Optimization for Mobile Robots,” International Journal of Precision Engineering and Manufacturing, vol. 24, no. 3, pp. 435–446, Jan. 2023.
26. S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” The International Journal of Robotics Research, vol. 30, no. 7, pp. 846–894, Jun. 2011.
27. H. Liu, S. Zhang, Y. Duan, W. Jia, and Y. Shen, “Orchard Robot Motion Planning Algorithm Based on Improved Bidirectional RRT,” Transactions of the Chinese Society of Agricultural Machinery, vol. 53.
28. H. Tu, Y. Deng, Q. Li, M. Song, and X. Zheng, “Improved RRT global path planning algorithm based on Bridge Test,” Robotics and Autonomous Systems, vol. 171.
29. L. Bin, F. Wan, Y. Hua, R. Ma, S. Zhu, and X. Qing, “F-RRT*: An improved path planning algorithm with improved initial solution and convergence rate,” Expert Systems with Applications, vol. 184, pp. 115457–115457, Dec. 2021.
30. A. Iskander, O. Elkassed, and A. El-Badawy, “Minimum Snap Trajectory Tracking for a Quadrotor UAV using Nonlinear Model Predictive Control,” 2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES), Oct. 2020.
31. Marcelino, Rahul Moghe, and M. R. Akella, “Real-Time Minimum Snap Trajectory Generation for Quadcopters: Algorithm Speed-up Through Machine Learning,” May 2019.